The Stardust Physics Engine: A Complete Beginner's Guide

Welcome! This guide is designed to take you from knowing absolutely nothing about programming to understanding every piece of the **Stardust Physics Engine**.

Stardust is a 3D gravity simulation program. It creates a tiny digital universe where planets orbit a sun, pull on each other with gravity, and even crash into each other, exploding into tiny fragments! You can fly around this universe with your keyboard and mouse, pause time, and tweak the planets.

Table of Contents

This document is split into multiple "pages" (separated by horizontal lines). We will start with the absolute basics of programming and then walk through your code file by file.

Page 1: Programming & C++ Basics

Before we look at the simulation, you need to understand the language it's written in: **C++**. Think of code as a recipe. The computer is a very fast, very obedient, but completely literal-minded chef. You have to tell it exactly what to do.

Here are the core concepts you will see everywhere in the project:

1. Variables and Data Types

A **variable** is like a labeled box where we store information. Every box has a **data type** that tells the computer what kind of information goes inside.

- `int` **(Integer):** Whole numbers (e.g., `5`, `-10`).
- `float` **(Floating-point):** Numbers with decimals (e.g., `3.14`).
- `bool` **(Boolean):** A simple true or false (e.g., `true`).
- `std::string`**:** Text, like words or sentences (e.g., `"Earth"`).
- `Vector3`**:** A special box that holds three `float`s (X, Y, and Z). Perfect for 3D space!
- `Color`**:** A special box holding Red, Green, Blue, and Alpha (transparency) values.

2. Functions

A **function** is a mini-machine or a specific "step" in our recipe. You put ingredients in (called **parameters**), the machine does some work, and it might spit out a result (called the **return value**). For example, a function called `ApplyGravity` takes two planets as ingredients and changes their speeds.

3. Loops and Conditionals

- **`if` / `else`:** This is how the computer makes decisions. "IF the distance between two planets is 0, THEN they collide, ELSE do nothing."
- **`for` loops:** This tells the computer to repeat a task. If we have 10 planets, a `for` loop says: "Run this gravity math 10 times, once for each planet."

4. Structs (and Classes)

A `struct` is a custom super-box you design yourself. If you want to define a `Planet`, you don't want to keep track of its mass, radius, and color separately. You create a `struct Planet` that holds all these variables inside one neat package.

5. std::vector

Do not confuse this with `Vector3` (which is a 3D arrow/coordinate). In C++, `std::vector` is a **dynamic list** or a train of boxes. If you have an unknown number of fragments from an explosion, you put them in a `std::vector<Fragment>`. You can `push_back()` to add a new fragment to the end of the train, and the train magically gets longer.

6. Pointers and References (`*` and `&`)

Normally, when you pass a variable to a function, the computer copies it. If the function modifies it, the original is untouched.

- **Reference (`&`):** This is a nickname or a direct wire to the original box. `void UpdatePosition(Planet &body)` means "don't copy the planet; modify the actual planet directly."
- **Pointer (`*`):** This is a piece of paper with the *address* of the box. `Planet *selectedPlanet` means "I am pointing to the address of the planet the user clicked on." If it equals `nullptr`, it's pointing to nothing.

Page 2: The Core Data (Planet.h)

In C++, an `.h` file is a "Header" file. It acts as a table of contents or a blueprint. The `.cpp` file contains the actual meaty instructions.

Let's look at `Planet.h`. This file's only job is to describe what a "Planet" is.

```
#pragma once
#include "raylib.h"
#include <string>

// Celestial body data structure
struct Planet {
  Vector3 position;
```

```
    Vector3 velocity;
    float mass;
    float radius;
    std::string modelPath;
    Color tint;
    float rotationAngle;
    float rotationSpeed;
    bool isAlive;
    std::string name;
    float massMin, massMax;
    float radiusMin, radiusMax;

    // This is a "Constructor". It's the setup machine for a New Planet.
    Planet(Vector3 pos, Vector3 vel, float m, float r, std::string path, Color c,
           float rotSpeed, std::string n,
           float mMin, float mMax, float rMin, float rMax)
        : position(pos), velocity(vel), mass(m), radius(r), modelPath(path),
    tint(c),
          rotationAngle(0.0f), rotationSpeed(rotSpeed), isAlive(true),
          name(n), massMin(mMin), massMax(mMax), radiusMin(rMin), radiusMax(rMax) {}
    };
```

What does this do?

- `#pragma once`: Tells the compiler "only read this file once, even if I include it multiple times."
- `struct Planet`: We are defining our custom data box.
- **Properties:** Every planet has a 3D `position`, a `velocity` (how fast and in what direction it's moving), `mass` (how heavy it is), `radius` (how big it is), and an `isAlive` flag (if it gets destroyed, we set this to `false`).
- **The Constructor (`Planet(...)`):** This block of code runs whenever a new planet is born. It takes a bunch of starting values (`pos`, `vel`, `m`) and assigns them to the internal variables (`position`, `velocity`, `mass`).

Page 3: The Physics Engine (Physics.h & Physics.cpp)

Now that we have planets, we need rules for how they move.

`Physics.h` simply lists our two math tools: `ApplyGravity` and `UpdatePosition`. `Physics.cpp` is where the magic happens. Let's walk through it.

1. ApplyGravity (Line-by-Line)

This function uses **Newton's Law of Universal Gravitation**. It calculates how hard `attractor` pulls on `target`.

Note on Gravity Strength: In our engine, we pass `G = 1.0f`. In the real universe, `G` is an unimaginably tiny number (6.674×10^{-11}). But since Stardust is designed to be a fun, visually pleasing simulation on your screen, all our numbers are scaled for gameplay. Using `1.0` keeps the math simple and makes the planets orbit beautifully!

```
void ApplyGravity(Planet &attractor, float attractorMass, Planet &target, float G,
float dt) {
```

We take the `attractor` (pulling), the `target` (being pulled), `G` (gravity strength, which is `1.0`), and `dt` (Delta Time - the fraction of a second since the last frame).

```
    Vector3 direction = Vector3{attractor.position.x - target.position.x,
                                attractor.position.y - target.position.y,
                                attractor.position.z - target.position.z};
```

We subtract the target's position from the attractor's position. This gives us a 3D arrow pointing directly from the target to the attractor.

```
    float distanceSquared = (direction.x * direction.x) +
                            (direction.y * direction.y) +
                            (direction.z * direction.z);
```

Using the Pythagorean theorem inside a 3D space, we calculate the distance squared. (We don't square root it yet because Newton's formula actually requires the squared distance!).

```
    if (distanceSquared > 0.0001f) {
```

Singularity guard: If two planets are in the exact same spot, distance is 0. Dividing by 0 crashes computers! We only apply gravity if there's a tiny bit of distance.

```
      float distance = std::sqrt(distanceSquared);
      Vector3 normalizedDirection = Vector3{
          direction.x / distance, direction.y / distance, direction.z / distance};
```

We find the exact distance, then divide our `direction` arrow by that distance. This gives us a "Normalized" vector—an arrow with a length of exactly 1.0, purely representing direction, without distance mixed in.

```
      float force = G * (attractorMass * target.mass) / distanceSquared;
      float acceleration = force / target.mass;
```

Here is Newton's formula: Force equals `G` times (mass 1 times mass 2) divided by distance squared. Then, we use $F = ma$ (Force = mass × acceleration) rearranged as $a = F/m$ to find out how much the target accelerates.

```
    Vector3 velocityChange = Vector3{normalizedDirection.x * acceleration * dt,
                                     normalizedDirection.y * acceleration * dt,
                                     normalizedDirection.z * acceleration * dt};

    target.velocity.x += velocityChange.x;
    target.velocity.y += velocityChange.y;
    target.velocity.z += velocityChange.z;
  }
}
```

Finally, we multiply our pure 1.0 direction arrow by our `acceleration`, and by `dt` (so it runs smoothly over time). We add this change to the target's current velocity. The target is now falling toward the attractor!

2. UpdatePosition (Euler Integration)

```
void UpdatePosition(Planet &body, float dt) {
  body.position.x += body.velocity.x * dt;
  body.position.y += body.velocity.y * dt;
  body.position.z += body.velocity.z * dt;
}
```

This is called "Kinematic update" or "Euler Integration". It simply means: New Position = Old Position + (Speed × Time). We do this every frame to move the planet slightly along its path.

Page 4: Collisions and Explosions (Collision.h & Collision.cpp)

What happens when two planets touch? We have to merge them and spawn debris!

`Collision.h` defines a `Fragment` struct. Think of a fragment as a miniature, temporary planet that slowly fades away (handled by the property `float life; // 1.0 to 0.0`).

ProcessCollisions Walkthrough

```
void ProcessCollisions(std::vector<Planet> &activePlanets,
                       std::vector<Fragment> &activeFragments,
                       Planet *&selectedPlanet, bool &isTracking) {
```

We provide the central lists: all active planets, all active fragments, what the user has currently clicked on (`selectedPlanet`), and whether the camera is attached to it (`isTracking`).

```
  for (size_t i = 0; i < activePlanets.size(); i++) {
    for (size_t j = i + 1; j < activePlanets.size(); j++) {
```

These nested loops check every planet against every *other* planet exactly once.

```
      float distSqr = Vector3DistanceSqr(activePlanets[i].position,
activePlanets[j].position);
      float combinedRadii = activePlanets[i].radius + activePlanets[j].radius;
      float collisionThreshold = combinedRadii * 0.8f;
```

Here we check if the squared distance between their centers is less than our collision threshold. Notice the `0.8f` (80%) multiplier! This means the planets don't merge the millisecond they graze one another; they must overlap by at least 20% of their combined radii. This ensures they visibly smash deep into each other before the visual explosion triggers.

```
      if (distSqr < (collisionThreshold * collisionThreshold)) {
        // Find survivor and victim
        ... (Code determines which is bigger. Sun is always survivor 0) ...
```

The code checks which planet is heavier. The heavy one is the `survivor`, the light one is the `victim`.

```
        victim.isAlive = false;
        if (selectedPlanet == &victim) {
          selectedPlanet = nullptr;
          isTracking = false;
        }
```

The victim is killed (`isAlive = false`). If the player was watching the victim, we break the camera lock so it doesn't crash trying to follow a ghost.

```
        // Momentum math
        Vector3 p1 = Vector3Scale(survivor.velocity, survivor.mass);
        Vector3 p2 = Vector3Scale(victim.velocity, victim.mass);
        Vector3 totalMomentum = Vector3Add(p1, p2);
        survivor.velocity = Vector3Scale(totalMomentum, 1.0f / totalMass);

        survivor.mass += victim.mass;
```

Conservation of Momentum: When things crash in space, they don't just stop. Let's say a fast, tiny asteroid hits a slow, massive planet. The planet absorbs the asteroid's mass, and its speed changes slightly based on the momentum (mass × velocity) of the asteroid. We also recalculate the survivor's new volume to make it grow physically larger.

```
        int numFragments = 30 + GetRandomValue(0, 20);
        for (int f = 0; f < numFragments; f++) {
            // ... fragment generation ...
```

```
            activeFragments.push_back(frag);
        }
    }
```

Finally, we generate a random number of rock fragments (between 30 and 50). We assign them a random direction, a speed based on the victim's old speed, and push them into the `activeFragments` list to be drawn on screen.

Page 5: Player Input and Cameras (Input.h & Input.cpp)

`ProcessDesktopInput` handles keyboard and mouse instructions.

```
void ProcessDesktopInput(...)
```

1. **Zooming:** `GetMouseWheelMove()` reads the scroll wheel to adjust `cameraSpeed`, letting the user fly faster or slower.
2. **Looking Around:**

   ```
   if (IsMouseButtonDown(MOUSE_BUTTON_RIGHT)) { ... }
   ```

   If you hold the Right Mouse Button, it reads `GetMouseDelta()` (how far you dragged your mouse) and applies math to rotate the camera's "forward" direction.
3. **Flying (WASD):** When you press `W`, `A`, `S`, or `D`, the code takes the camera's current position and adds your `cameraSpeed` to push you forward, back, left, or right. If you move manually, `isTracking = false` prevents the camera from snapping back to a planet.
4. **Clicking Planets (Mouse Picking):**

   ```
   Ray mouseRay = GetMouseRay(mpos, camera);
   ```

   When the player left-clicks, the engine casts an invisible laser (`Ray`) from the screen directly into the 3D world. It checks if this laser intersects with the spheres (planets). The closest hit becomes the `selectedPlanet`. (The code also makes sure you aren't clicking on the UI panels before casting this ray.)

Page 6: Heads Up Display / UI (HUD.h & HUD.cpp)

This file manages the 2D shapes and text superimposed on the screen.

DrawSelectionReticle

When a planet is selected, `DrawSelectionReticle` projects the planet's 3D position onto the 2D screen (`GetWorldToScreen`). It then draws four rotating 2D arcs (`DrawRing`) and target lines, giving it a sci-fi "Target

Locked" appearance.

DrawDebugOverlay

This uses a library called `raygui.h`. You do not need to understand raygui's inner code (it is thousands of lines long!), but here is how Stardust *uses* it:

- `DrawRectangleRounded` simply draws semi-transparent black boxes for backgrounds.
- `DrawText` drops standard text onto the screen (like "Mass: 50.0").
- `GuiSlider(...)` draws a slider on screen. Notice we pass `&selectedPlanet->mass`. Remember the `&` (reference/pointer)? The slider directly reaches into the planet memory box and changes the mass as you drag it!
- `GuiButton(...)` draws a clickable button. If the user clicks it, it acts like a true/false statement, toggling the `isTracking` variable.

At the very bottom, it counts how many planets are alive and shows your Frames Per Second (FPS).

Page 7: Bringing It All Together (main.cpp)

`main.cpp` is the heart of the engine. In C++, the `int main()` function is the exact starting point of the program.

1. Initialization

```
InitWindow(screenWidth, screenHeight, "Stardust");
```

This tells our graphics library (raylib) to pop open a window on your monitor. We then set up the 3D camera, initialize our lists of data (`std::vector`), and load a 3D shader to make things look pretty (using `rlights.h` to create a virtual sun).

Audio/Music: Before the simulation runs, we also prepare the audio by calling `InitAudioDevice()`. We load a `MusicStream` for `ambient_space.mp3`, set it to loop smoothly, and drop its volume down to `0.025`. This keeps the music playing as a very soft, faint hum in the background so that space doesn't feel totally silent.

2. Creating the Solar System

```
initialPlanets = {
    Planet({0, 0, 0}, {0, 0, 0}, 2000, 3, "assets/sun.glb", ...),
    Planet({8, 0, 8}, {-15, 0, 15}, 0.05, 0.25, "assets/mercury.glb", ...),
    ...
```

We build the solar system. We give each planet a start coordinate, an initial push (velocity), mass, size, and load a `.glb` 3D model file for it. *Note:* The code immediately does some momentum math to make sure the Sun wobbles slightly based on the pull of the planets, keeping the system balanced!

3. The Game Loop

```
while (!WindowShouldClose()) {
```

This loops roughly 144 to 180 times *every second*. Everything inside this loop is one "frame" of a video game.

- `dt = GetFrameTime();` grabs the time since the last frame (e.g., 0.006 seconds). It keeps the game running at the same speed regardless of how fast your computer is.
- `UpdateMusicStream(ambientMusic);` continually weaves in the next piece of the audio track.
- We check if the user hits `SPACE` to pause or `R` to reset the universe.
- `ProcessDesktopInput(...)` updates user movement.
- If the camera is tracking a planet (`isTracking`), physics math swoops the camera behind the planet using a smoothed math function `Vector3Lerp` (Linear Interpolation).

4. Running the Physics

```
if (currentState == PLAYING) {
  const int SUB_STEPS = 10;
  for (int step = 0; step < SUB_STEPS; step++) {
```

Gravity is calculated at 10 "sub-steps" per frame. Because planets accelerate drastically when they get close to each other, running the math 10 tiny times is much more accurate than 1 big time. Inside this loop, it calls `ApplyGravity` and `ProcessCollisions`.

5. Drawing to the Screen

```
BeginDrawing();
ClearBackground(BLACK);
BeginMode3D(camera);
```

We wipe the old frame clean. We switch into 3D mode. A loop goes over all `activePlanets` and uses `DrawModelEx` to draw their 3D models.

Shader Swapping for the Sun: Notice how we treat the Sun (which is index `0` in our array)! Because the Sun is the actual object *creating* the light in our solar system, we do not want it to be darkened by its own shadows. Right before drawing it, we swap its material to a `defaultSunShader` (an unlit, basic material) so it always appears perfectly bright. After we draw it, we re-apply the complex light shader so the rest of the planets are shaded correctly.

Another loop draws little cubes (`DrawCubeV`) for the explosion fragments. The fragments' `life` timer ticks down, making them fade out. Dead fragments are completely deleted (`erase`/`remove_if`).

End 3D mode. Call the UI functions (`DrawDebugOverlay`). `EndDrawing();` pushes the new frame to your actual monitor.

This loops endlessly until you hit the X button on the window. We then run cleanup functions (`CloseWindow`) and the program finishes successfully (`return 0`).

Page 8: How to Build & Run

Dependencies

Stardust uses a few libraries to do the heavy lifting:

1. **raylib:** An incredibly popular, easy-to-use library that handles window creation, mouse inputs, audio, and drawing 3D shapes.
2. **raygui.h:** An extension to raylib (used as a single massive file) that gives you immediate sliders and buttons.
3. **rlights.h:** A helper script that feeds lighting information from our code to the graphics card (shaders).

Folder Structure

For this to run without crashing, the compiled executable (`Stardust.exe`) needs to be in a folder alongside your assets. The layout looks like this:

```
Stardust_Folder/
|-- Stardust.exe
|-- assets/
|   |-- sun.glb
|   |-- earth.glb
|   |-- (and all other .glb models)
|   |-- ambient_space.mp3
|-- resources/
|   |-- shaders/
|       |-- glsl330/
|           |-- lighting.vs
|           |-- lighting.fs
```

Compiling (Using Visual Studio on Windows)

1. Open the file `Stardust.vcxproj` using Microsoft Visual Studio.
2. The project uses NuGet Packages to grab its dependencies (you can see `packages.config` in your folder). Visual Studio will automatically download `raylib` for you via NuGet.
3. At the top of Visual Studio, set exactly two dropdowns: `Debug` or `Release`, and `x64` (64-bit).
4. Hit **F5** (Local Windows Debugger) to build the engine and launch Stardust!

Page 9: Glossary

- **Acceleration:** How quickly something is speeding up, slowing down, or changing direction.
- **Camera:** A mathematical concept in code that defines what the player "sees" on their 2D monitor from a fake 3D world.
- **Class/Struct:** A blueprint for an object. Like a template for a "Planet" that defines what it holds.
- **Collision Detection:** Math that checks if two objects in 3D space are overlapping.
- **Delta Time (`dt`):** The tiny sliver of time (usually ~0.016 seconds) that passed since the computer drew the last frame. It's multiplied against speeds to ensure smooth movement.

- **Euler Integration:** A basic numerical method used in physics engines. You simply say `position = position + velocity * time`.
- **Force:** A push or a pull. Gravity is a force.
- **Fragment:** In Stardust, the visual debris spawned when planets collide.
- **Gravity:** An attractive force. Newton's law states that every object pulls on every other object, and it gets stronger the heavier they are, and weaker the further apart they are.
- **Impulse/Momentum:** Mass times velocity. Used here to figure out how fast a giant combined planet moves after a crash.
- **Pointer/Reference:** A programming tool that acts like an address. Instead of copying a giant block of data, you just hand the computer a sticky note saying "The data is over there."
- **Shader:** A mini-program specifically written for your Graphics Card (GPU). `lighting.vs` and `.fs` tell the graphics card how to color pixels so things look round and sunlit, rather than flat.
- **Vector (`std::vector`):** The C++ term for a list that can grow and shrink in size.
- **Vector (Physics/Math):** Represented by `Vector3`. A quantity with both magnitude (size) and direction. A 3D coordinate (x, y, z) functions as a mathematical vector.
- **Velocity:** Speed combined with a direction. "50 mph" is speed. "50 mph pointing strictly upward" is velocity.